29、spi-nor image制作

全志V3S嵌入式驱动开发（spi-nor image制作）

原创

嵌入式-老费Open: 7f185b501ef38a059b10d1b79d7cfdb9_MD5.png

已于 2023-08-29 15:18:23 修改

Open: 8a923f94c34ac151897c6be747452d7c_MD5.png

阅读量879 Open: 196806fd3bdd4f04e1a92a7697cf0354_MD5.png

收藏 1

Open: 95eb4f436541fdf1b2e1af49844ad0c3_MD5.png

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

Open: 135c7190ea3ccdc82fa1f0c0a41d08f0_MD5.png

基于全志soc的linux嵌入式开发专栏收录该内容](https://blog.csdn.net/feixiaoxing/category_12270221.html "基于全志soc的linux嵌入式开发")

73 篇文章¥49.90¥99.00

已订阅8折续费

[

Open: d9b05f322fadabcd81dc6c5654189e59_MD5.png

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益Open: ddf4d655e9aa3806864ef10c7c4a060b_MD5.png

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

Open: 1e4395309f4514fed3e1f5e89b2a25e6_MD5.png

摘要 本文介绍了全志V3S嵌入式驱动开发过程，特别是如何创建和烧录spi-nor镜像。内容包括根文件系统bin文件的创建、uboot代码的修改、kernel的调整、uboot的初步尝试、完整镜像的构建以及烧入flashimg.bin到开发板。整个流程详细解释了每个步骤，并提供了关键配置和命令。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

其实，我们之前就讨论过怎么把image烧入到v3s的spi-nor当中去。当时使用的方法是借助于sunxi-fel工具，烧入的image也比计较小，只是一个uboot bin文件。今天，我们就来讨论一下，一个完整的spi-nor镜像应该如何创建和烧入。

1、创建根文件系统bin文件

关于根文件系统如何编译，之前在buildroot那篇文章已经讲过。这里讨论的是，怎么把根文件系统压缩成镜像文件。一个完整的镜像文件根本上取决于spi-nor的大小。因为我们选择的存储芯片是MX25L25645G，自身大小是32M，所以给rootfs的空间其实可以这么来安排，

即0x0~0×100000是uboot空间，0x100000~0×110000是dtb空间，0x110000~0×610000是kernel空间，剩下来的0x610000~都是rootfs的空间。所以rootfs的大小是，

```
32M-1M-64K-5M=0x19F0000
```

创建image过程中涉及到mtd-utils工具安装，

```
sudo apt-get install mtd-utils
```

在用mtd-utils创建image之前，一般先修改下rootfs目录下面文件的权限，不然启动后执行有问题，

```
sudo chown root * -R
```

设置完权限之后就可以创建镜像文件了，我们选择的文件系统是jffs2，其中0x100表示一个page大小，0x10000表示一个sector大小，0x19f0000表示镜像大小，

```
sudo mkfs.jffs2 -s 0x100 -e 0x10000 -p 0x19F0000 -d target/ -o
jffs2.img
```

注意实际生成的jffs2.img不等于0x19F0000，它还需要copy到总的flashimg.bin当中去。0x19F0000只是表示它可以占领这么大的空间。

2、修改uboot代码

uboot选用的还是v3s-spi-experimental这个分支，

```
https://github.com/Lichee-Pi/u-boot/tree/v3s-spi-experimental
```

uboot最主要的工作就是修改头文件，位于 include/configs/sun8i.h，在“#include <configs/sunxi-common.h> ”的前边添加，

```
#define CONFIG_BOOTCOMMAND   "sf probe 0; "                          \
                            "sf read 0x41800000 0x100000 0x10000; "  \
                            "sf read 0x41000000 0x110000 0x500000; " \
                            "bootz 0x41000000 - 0x41800000"

#define CONFIG_BOOTARGS      "console=ttyS0,115200 earlyprintk panic=5
rootwait " \

"mtdparts=spi0.0:1M(uboot)ro,64k(dtb)ro,5M(kernel)ro,-(rootfs)
root=31:03 rw rootfstype=jffs2"
```

这里面内容比较多，我们可以依次介绍下。

sf probe 0表示选择第一个 nor flash；

sf read 0×41800000 0×100000 0×10000表示从flash 0×100000的位置拷贝64k到内存0x41800000的位置；

sf read 0×41000000 0×110000 0×500000表示从flash 0×110000的位置拷贝5M到内存0x41000000的位置；

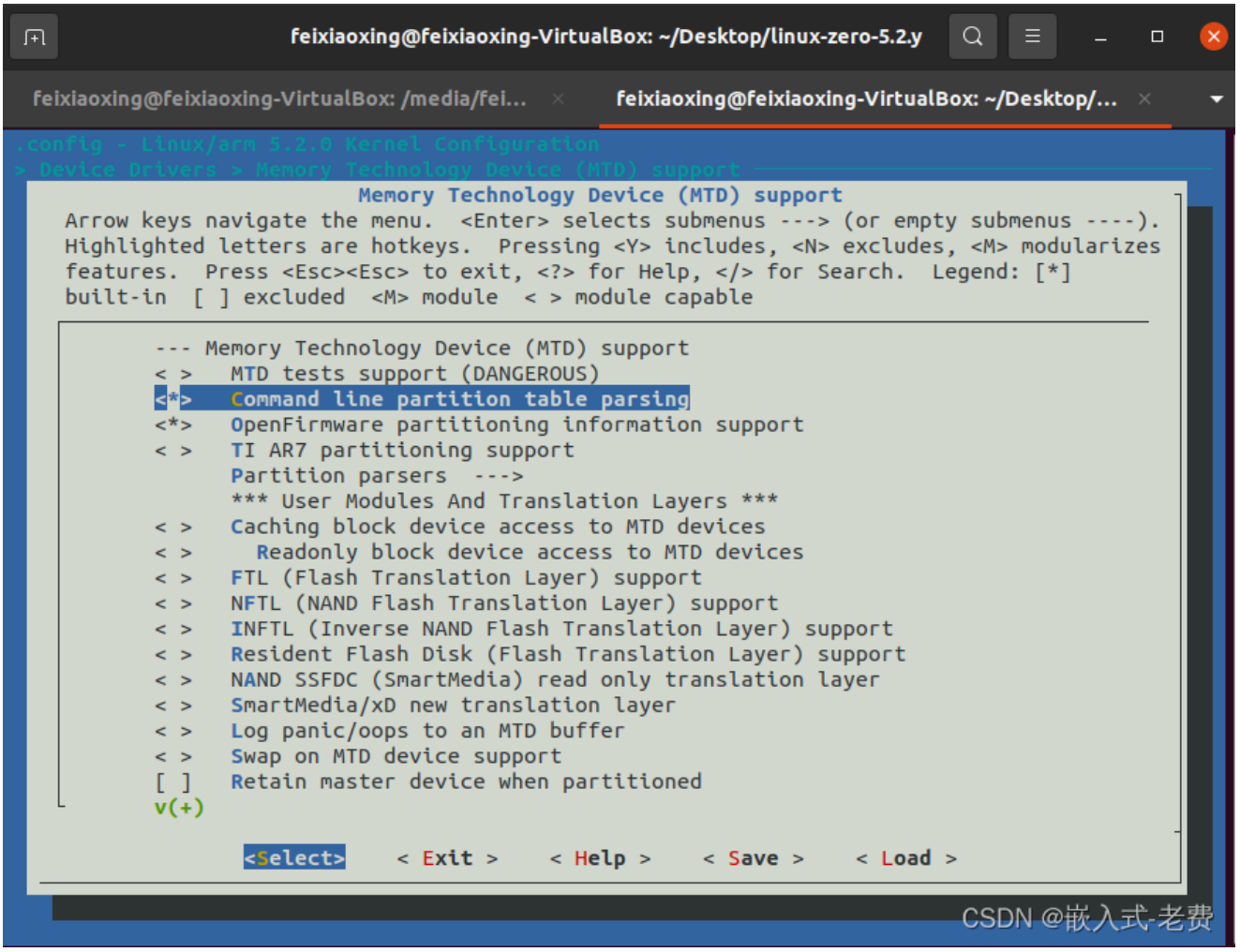
bootz 0×41000000 - 0×41800000表示告诉cpu kernel和dtb位于这两个位置，稍后cpu会跳到kernel的位置继续执行。

CONFIG_BOOTARGS稍微复杂点，里面有几个地方需要注意下。 **第一，mtdparts中间一定要写成spi0.0，不能写成spi32766.0，老的kernel版本，比如4.14.y也许可以这样写，但是5.2.y的新kernel一定要写成spi0.0形式；** 第二，mtdparts中各个部分布局一定要和之前规划的大小一致； **第三，kernel的大小一定是5M空间，新的kernel用4M空间已经装不下了。**

3、修改kernel

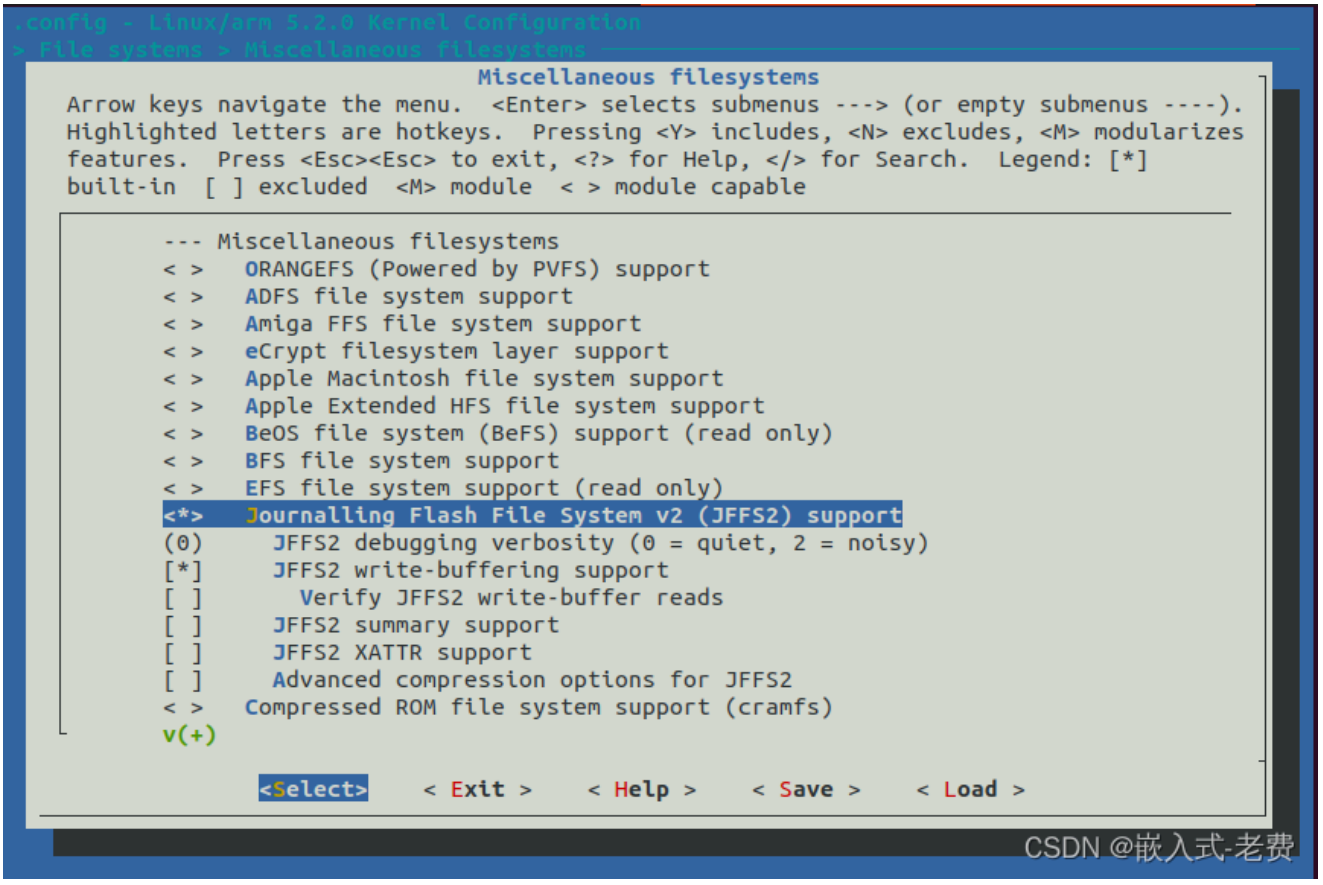
kernel中有两个地方需要添加一下配置。第一个是“Command line partition table parsing”，

Open: b727187ae8092ac20f7387cfbd7f3329_MD5.png



另外一个，就是“ Journalling Flash File System v2 (JFFS2) support”，它的主要功能就是让kernel支持jffs2文件系统，

Open: 715cdf4928b3a6c68a6aab8d52b46e7b_MD5.png



有了这两个配置，重新输入make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-即可。spi的部分，复用之前spi-nor驱动的修改就行，不需要修改设备树。

4、uboot初步尝试

此时，我们已经准备好了uboot、kernel、dtb和jffs2.img，似乎可以奔着最终的目标去了。 **不过这里，还是建议大家先借助于uboot+y modem的方法确认kernel和dtb为运行是否ok，没问题了再继续推进。** 首先还是先用sunxi-fel烧入uboot，等uboot起来之后，借助于uboot下的loady命令将kernel和dtb拷贝到内存当中，再借助于sf erase和sf write将内存中的数据写入flash，这样就可以开始测试uboot了，依次输入命令，

```
sf probe 0;
sf read 0x41800000 0x100000 0x10000;
sf read 0x41000000 0x110000 0x500000;

setenv bootargs console=ttyS0,115200 panic=5 rootwait
mtdparts=spi0.0:1M(uboot)ro,64k(dtb)ro,5M(kernel)ro,-(rootfs)
root=31:03 rw rootfstype=jffs2;

bootz 0x41000000 - 0x41800000
```

这部分内容和uboot代码修改最大的区别就是setenv命令，当然只是为了满足uboot交互输入的需要。做好了这几个步骤，基本上uboot可以顺利进入kernel了，只是暂时没办法加载根文件系统而已，如下面打印所示，

```
[    1.408568] VFS: Cannot open root device "31:03" or unknown-
block(31,3): error -19
[    1.416257] Please append a correct "root=" boot option; here are
the available partitions:
[    1.424673] Kernel panic - not syncing: VFS: Unable to mount root fs
on unknown-block(31,3)
[    1.433020] CPU: 0 PID: 1 Comm: swapper/0 Not tainted 5.2.0-
licheepi-zero #10
[    1.440143] Hardware name: Allwinner sun8i Family
[    1.444877] [<c010ede4>] (unwind_backtrace) from [<c010b764>]
(show_stack+0x10/0x14)
[    1.452618] [<c010b764>] (show_stack) from [<c06e1f98>]
(dump_stack+0x88/0x9c)
[    1.459837] [<c06e1f98>] (dump_stack) from [<c011dce4>]
(panic+0x110/0x2fc)
[    1.466797] [<c011dce4>] (panic) from [<c09012ec>]
(mount_block_root+0x218/0x2f4)
[    1.474273] [<c09012ec>] (mount_block_root) from [<c0901594>]
(prepare_namespace+0x144/0x188)
[    1.482790] [<c0901594>] (prepare_namespace) from [<c06f86c8>]
(kernel_init+0x8/0x10c)
[    1.490701] [<c06f86c8>] (kernel_init) from [<c01010e8>]
(ret_from_fork+0x14/0x2c)
[    1.498260] Exception stack(0xc3833fb0 to 0xc3833ff8)
[    1.503306] 3fa0:                                     00000000
00000000 00000000 00000000
[    1.511474] 3fc0: 00000000 00000000 00000000 00000000 00000000
00000000 00000000 00000000
[    1.519640] 3fe0: 00000000 00000000 00000000 00000000 00000013
00000000
[    1.526261] Rebooting in 5 seconds..
```

5、构建完整的镜像文件

等uboot加载kernel没有问题之后，就可以着手构建完整的镜像文件了。因为我们之前都已经准备好了uboot、kernel、dtb和jffs2.img，那么再准备一个镜像生成的脚本就

可以了，当然首先需要把它们这些文件都copy到同一个目录下面，

```
dd if=/dev/zero of=flashimg.bin bs=1M count=32
dd if=u-boot-sunxi-with-spl.bin of=flashimg.bin bs=1K conv=notrunc
dd if=sun8i-v3s-licheepi-zero-with-480x272-lcd.dtb of=flashimg.bin
bs=1K seek=1024  conv=notrunc
dd if=zImage of=flashimg.bin bs=1K seek=1088  conv=notrunc
dd if=jffs2.img of=flashimg.bin  bs=1K seek=6208  conv=notrunc
```

脚本共五条命令。第一条，创建一个32M的空白文件flashimg.bin。第二条，导入uboot。第三条导入dtb。第四条，导入kernel zImage。第五条，导入根文件系统jffs2.img。最后我们需要烧入的就是flashimg.bin这个文件。

6、烧入flashimg.bin和启动v3s

前面我们说过，单纯用sunxi-fel烧入大文件比较容易失败。 **一个比较务实且快速的办法是，首先利用uboot里面的sf erase命令erase掉整个flash；然后用sd卡启动v3s后，利用dd命令copy flashimg.bin文件。**

第一步，uboot在erase整个flash的时候，不需要担心uboot自身是不是会崩溃。因为，uboot自己是运行在ddr空间里面，而erase的是norflash，

```
sf erase 0x0 0x2000000;
```

第二步，等v3s用sd卡启动linux之后，norflash就被当成linux的一个外设，所以直接用dd拷贝flashimg.bin即可，它的速度是sunxi-fel的几十倍，还很稳定，

```
dd if=flashimg.bin of=/dev/mtd0
```

做好这一切之后，就可以拔掉sd卡，准备重启v3s开发板了，不出意外，就可以看到v3s顺利启动的打印信息了，甚至看一下mtd的布局，

```
[    1.411951] vcc3v0: disabling
[    1.414930] vcc3v3: disabling
[    1.417894] vcc5v0: disabling
[    1.420925] ALSA device list:
[    1.423892]   No soundcards found.
[    1.429620] random: fast init done
[    1.473493] random: crng init done
[    4.350019] VFS: Mounted root (jffs2 filesystem) on device 31:3.
[    4.357361] devtmpfs: mounted
```

```
[    4.361668] Freeing unused kernel memory: 1024K
[    4.366360] Run /sbin/init as init process
Starting logging: OK
Initializing random number generator... done.
Starting network: OK

Welcome to Buildroot
buildroot login: root
# ls -l
total 0
# ls -l /dev/mtd*
crw-------    1 root     root       90,   0 Jan  1 00:00 /dev/mtd0
crw-------    1 root     root       90,   1 Jan  1 00:00 /dev/mtd0ro
crw-------    1 root     root       90,   2 Jan  1 00:00 /dev/mtd1
crw-------    1 root     root       90,   3 Jan  1 00:00 /dev/mtd1ro
crw-------    1 root     root       90,   4 Jan  1 00:00 /dev/mtd2
crw-------    1 root     root       90,   5 Jan  1 00:00 /dev/mtd2ro
crw-------    1 root     root       90,   6 Jan  1 00:00 /dev/mtd3
crw-------    1 root     root       90,   7 Jan  1 00:00 /dev/mtd3ro
```

最后，再确认一下，留给客户的mtd空间大约还剩下多少，

```
# df
Filesystem           1K-blocks      Used Available Use% Mounted on
/dev/root                26560      5744     20816  22% /
devtmpfs                 27520         0     27520   0% /dev
tmpfs                    28032         0     28032   0% /dev/shm
tmpfs                    28032        20     28012   0% /tmp
tmpfs                    28032        16     28016   0% /run
```

32M的总空间，uboot留了1M，dtb可以忽略，kernel大约5M，rootfs大约6M，剩下来的空间也就是32M -1M -5M -6M= 20M的空间了。如果没有图片和视频的话，这个空间用来存储一些程序和配置文件的话，还是比较富余的。